



**MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA**

**Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering**

VREMERE ADRIAN, FAF-232

Report

Laboratory Work n.6.1

Behavioural Control with Finite State Machines — Button-LED

on Embedded Systems

Checked by:
Martîniuc Alexei, *univ. asist.*
FCIM, UTM

Chişinău – 2026

1. Domain Analysis

1.1. Objective of the Laboratory Work

The objective of Laboratory Work 6.1 is to introduce the design and implementation of finite state machines (FSM) in embedded systems, using the canonical Button-LED case study described in chapter "Lucrarea de laborator nr. 7: Control comportamental cu automate finite" of the laboratory manual. The application must read a momentary push button, debounce it in software, and toggle the state of an LED on every confirmed press, while reporting the current FSM state in real time on the STDIO serial interface.

The pedagogical goal is to model the behaviour explicitly as a Moore automaton with two states (LED off, LED on) and to encapsulate that behaviour in a dedicated software module, separate from the input and output drivers. This separation is what makes FSM-based design valuable: the same state table can be tested in isolation, reused with different actuators, or extended with additional states without rewriting the application.

The lab task statement, reproduced verbatim from the manual (section 7.2.1), is:

Se cere realizarea unei aplicații embedded care implementează un automat finit pentru controlul unui LED, în funcție de apăsarea unui buton. Sistemul va avea două stări: LED-ul stins (OFF) și LED-ul aprins (ON). Tranziția între aceste stări se va efectua la fiecare apăsare validă a butonului, cu implementarea logicii de debounce pentru eliminarea fluctuațiilor nedorite. Starea curentă a LED-ului va fi afișată în timp real prin interfața serială (printf) sau pe un LCD (2x16 sau 4x20), utilizând biblioteca STDIO sau LCD.

1.2. Problem Definition

The application targets an Arduino Mega 2560 development board and must satisfy the following functional and non-functional requirements derived directly from the task statement and the supplementary requirements ("Cerințe suplimentare") of the manual.

The functional requirements (RF) cover the input/output behaviour of the system. RF01: the application detects each press of the push button by reading its digital input. RF02: every confirmed press toggles the LED between the OFF and ON states; the transition is governed by a Moore FSM with exactly two states. RF03: the input is debounced in software so that mechanical contact bounce never produces multiple state transitions for a single physical press. RF04: the current FSM state is published in real time over the STDIO serial interface using the C standard library functions (`printf`, `fdev_setup_stream`).

The non-functional requirements (RNF) describe quality attributes. RNF01: the worst-case latency between a press and the corresponding state report is below 100 ms, which guarantees a perceptually instantaneous response. RNF02: the FSM logic is isolated in a software module (ButtonLedFsm) that can be unit-tested without any hardware in the loop. RNF03: all low-level peripheral access (pin configuration, digital reads, digital writes, UART setup) is encapsulated

inside reusable libraries under `labs/lib/`; no `digitalRead`, `digitalWrite`, `pinMode`, or direct `Serial` calls appear in the lab entry point. RNF04: the same compiled firmware runs identically in the Wokwi simulator and on physical hardware.

1.3. Used Technologies

Finite State Machines (Moore Model)

A finite state machine is a behavioural abstraction that describes a system as a finite set of stable states, a finite set of inputs, a transition function that maps "current state" together with "current input" to a "next state", and an output function. Two complementary flavours are commonly used: the Moore machine, where the output depends only on the current state, and the Mealy machine, where the output depends on both the current state and the current input. The Moore form is preferred when the output value must be visually or physically stable for as long as the system stays in the state, which is precisely the case for a glowing LED. A Mealy machine, by contrast, would react instantaneously to input changes, but its output could flicker if the input is noisy.

For the Button-LED problem, a Moore automaton with two states is sufficient. Following the structure proposed in the lab manual (Tabelul 7.1), each state is described by a row in a state-transition table that holds the Moore output value and the next-state lookup indexed by the input. Implementing the FSM as a constant table is valuable for two reasons: it removes ad-hoc `if/else` branching from the code, and it produces a one-to-one mapping between the formal state diagram and its source code representation, which simplifies review and testing.

Software Debounce

A push button is an electromechanical device whose contacts oscillate during a transition — after a press, the metal surfaces bounce against each other for a few milliseconds before settling. Without filtering, a microcontroller polling the input at MHz frequencies would interpret each bounce as a separate press and would multiply state transitions for a single physical action.

The classical mitigation is software debouncing with a stable-time filter. After every observed change in the raw input level, the driver records a timestamp and continues sampling the line. The change is accepted as a real edge only after the new level has remained stable for at least a configurable debounce window (typically 20–50 ms, far longer than any mechanical bounce yet much shorter than the human-perceptible threshold). This filter is implemented entirely in software and requires no extra hardware. It is the approach adopted by the `Button` driver in this lab, with a default window of 30 ms.

UART Serial Communication and STDIO

The Arduino Mega 2560 exposes `UART0` through the on-board USB-to-serial bridge, allowing a host PC to communicate with the microcontroller through any terminal application. UART is asynchronous: the two ends do not share a clock and must agree on a frame format (start bit, 8 data bits, no parity, 1 stop bit, 9600 baud is used here). At the firmware level, the C standard

library is redirected onto UART0 by attaching a custom FILE stream via `fdev_setup_stream()`, which lets the application produce diagnostics with `printf` instead of Arduino-specific calls. The same `StdioSerial` module developed in earlier labs is reused here.

1.4. Hardware Components

Arduino Mega 2560

The Arduino Mega 2560 is a development board built around the ATmega2560 8-bit AVR microcontroller, clocked at 16 MHz with 256 KB of Flash, 8 KB of SRAM, 4 KB of EEPROM, 54 digital I/O pins, 16 analog inputs, and four hardware UARTs. UART0 is wired to the on-board USB-to-serial converter, providing programming and serial monitoring through a single USB cable. For Lab 6.1 only two GPIO pins and UART0 are used, so the board's resources are barely loaded; the generous Flash and SRAM headroom leaves the firmware comfortably within budget (about 5.9 KB of Flash and 833 B of SRAM, as confirmed by the build output).

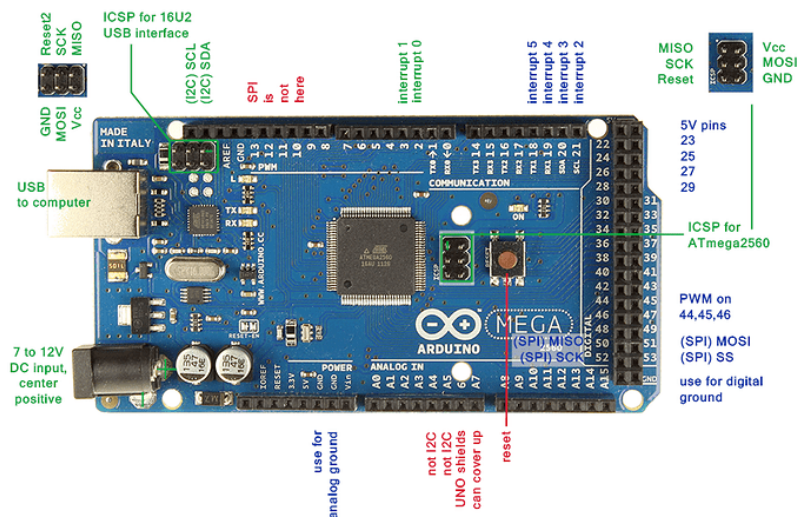


Figure 1.1 – Arduino Mega 2560 development board

Push Button

A standard tactile momentary push button is used as the system input. The two pins on one side are internally shorted to the two pins on the other side when the button is pressed; otherwise the contacts are open. The button is wired between digital pin 7 and ground, and the internal pull-up of the ATmega2560 is enabled by configuring the pin as `INPUT_PULLUP`. In this active-LOW configuration the pin reads HIGH when the button is idle and LOW while it is held. This avoids the need for an external pull-up resistor.



Figure 1.2 – Tactile push button used as the FSM input

LED and Current-Limiting Resistor

The output device is a standard 5 mm red LED with a forward voltage of approximately 2.0 V and a maximum forward current rating of 20 mA. It is connected between digital pin 8 and ground through a 220 Ω current-limiting resistor. Applying Ohm's law for the 5 V GPIO output gives a forward current of $(5\text{ V} - 2\text{ V})/220\Omega \approx 13.6\text{ mA}$, which is comfortably below both the LED rating and the per-pin limit of the ATmega2560.

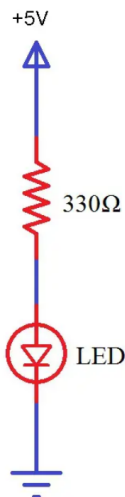


Figure 1.3 – Red LED used as the FSM Moore output

Breadboard and Jumper Wires

A solderless breadboard provides the physical prototyping platform; jumper wires connect the Arduino pins to the button and to the resistor-LED branch. No additional hardware is required.

1.5. Software Components

PlatformIO

PlatformIO is the build, dependency, and tooling layer used for this project. It manages the AVR toolchain, links the Arduino framework, exposes per-environment build flags (-DLAB6_1 selects the Lab 6.1 entry point), and orchestrates the upload and monitor steps. The `platformio.ini`

file declares an `[env:lab6_1]` environment that compiles only the Lab 6.1 sources together with the shared libraries under `labs/lib/`, ensuring that the previous labs remain runnable from the same project tree.

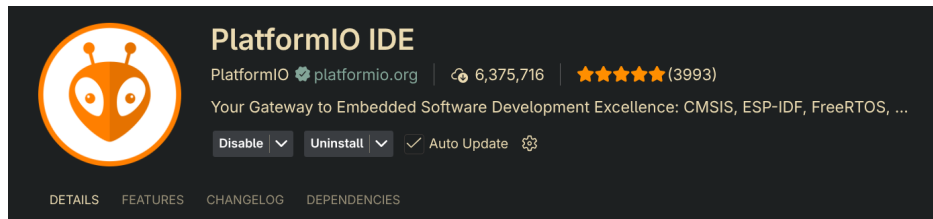


Figure 1.4 – PlatformIO IDE in VS Code

Wokwi

Wokwi is a circuit simulator that runs Arduino firmware against a virtual board, with virtual peripherals such as buttons, LEDs, resistors, and serial terminals. It integrates with VS Code through the official Wokwi extension, which reads the lab's `wokwi.toml` pointing to the compiled firmware and the `diagram.json` describing the circuit. Wokwi is used here to validate the FSM transitions, the debounce logic, and the STDIO output without requiring physical hardware to be connected.

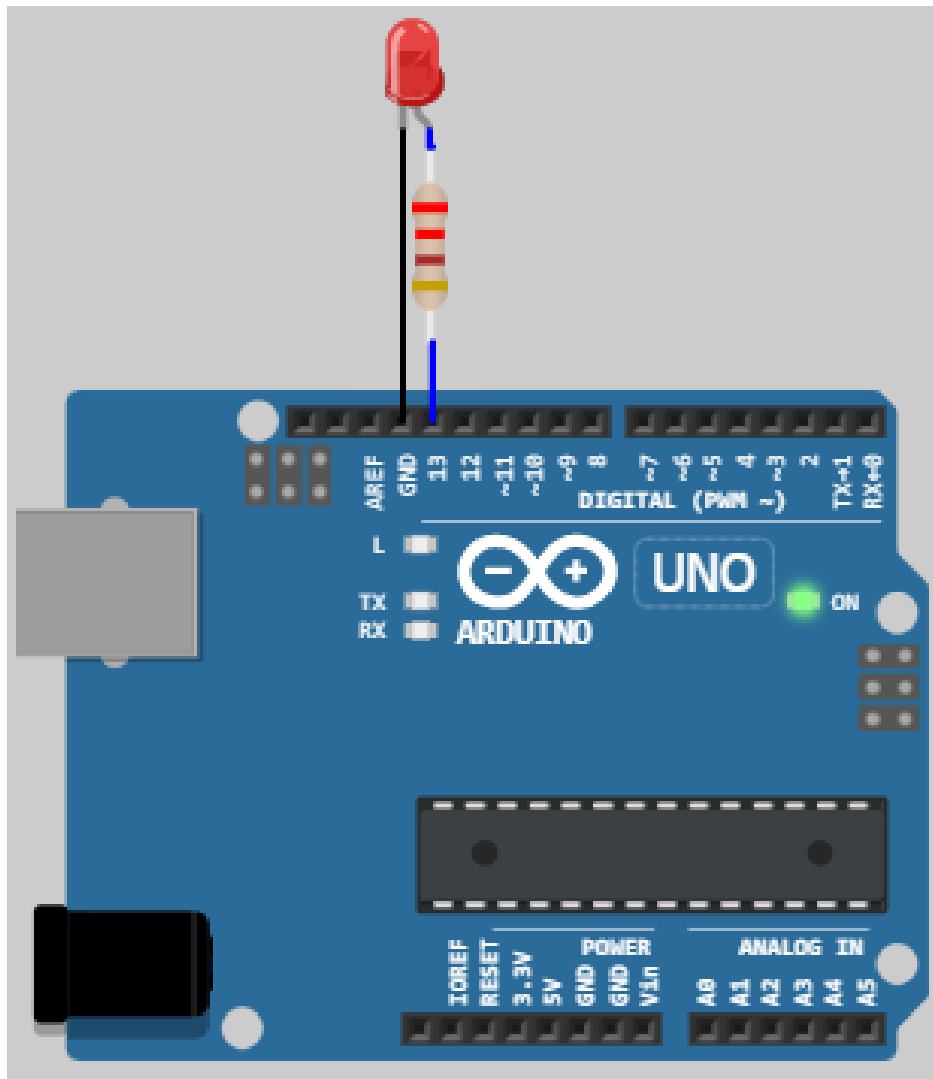


Figure 1.5 – Wokwi simulator running the Lab 6.1 circuit

1.6. System Architecture and Justification

The application follows the same layered architecture used throughout the course, with the FSM placed in the service layer. The Application Layer (APP) is the lab entry point `lab6_1_main.cpp`, whose only responsibility is to wire the drivers and the FSM together and to translate state changes into `STDIO` output. The Service Layer (SRV) hosts `ButtonLedFsm`, the hardware-independent Moore automaton; it knows nothing about pins or peripherals and could equally well drive a relay, an LCD line, or a remote endpoint. The ECU Abstraction Layer (ECAL) hosts `StdioSerial`, which redirects `stdout` onto `UART0` so that the application can use `printf`. The Microcontroller Abstraction Layer (MCAL) provides two device drivers: `Button`, which encapsulates pin configuration, digital reads, and the debounce state machine, and `Led`, which encapsulates the digital output. The Hardware (HW) layer is the ATmega2560 peripheral set itself: GPIO ports B and H for the LED and button respectively, and `USART0` for serial communication.

This decomposition is justified by two concerns. First, the lab manual explicitly requires the FSM logic to be separated from input/output drivers ("Separarea logicii FSM într-un modul software dedicat"), so that the state machine can be developed and tested independently. Second, the

project rule that no `digitalRead/digitalWrite/Serial` call may appear outside `labs/lib/` forces every hardware-related operation into a clearly named driver module, which keeps the lab entry point short and readable.

1.7. Case Study — Real-World Applicability

The Button-LED FSM is a minimal example, but the underlying pattern — "a debounced discrete input drives a finite-state controller that produces a stable physical output" — is one of the most pervasive idioms in embedded systems. Power buttons of consumer electronics implement exactly this two-state machine, sometimes with a long-press transition added for a third state ("force off"). Industrial pendant panels use FSMs to gate motor activation behind operator confirmation. Smart home wall switches that toggle relays through a single push button rely on the same logic, sometimes augmented with a network-driven external transition. In automotive electronics, many cabin functions (cruise-control engagement, hazard lights, parking brakes) are modelled as finite-state machines whose discrete inputs are debounced and whose outputs are stable Moore signals. The discipline of expressing such behaviour as a state table rather than as ad-hoc `if/else` branches is what allows these systems to be certified, formally verified, and safely extended over their long service lifetimes.

2. Design

2.1. System Architecture Diagrams

Component-Level Architecture

The Lab 6.1 system involves three physical components: the Arduino Mega 2560, a momentary push button wired to digital pin 7 with the internal pull-up enabled, and a current-limited LED driven from digital pin 8. The microcontroller exchanges diagnostic text with a host PC through UART0, which is exposed by the on-board USB-to-serial bridge. The structural diagram in *Figure 2.1* shows the data flow: a press event travels from the button through the GPIO peripheral and the `Button` driver into the FSM, the FSM updates its state and forwards the new Moore output to the `Led` driver and to the `STDIO` log, and the host terminal displays the resulting transition.

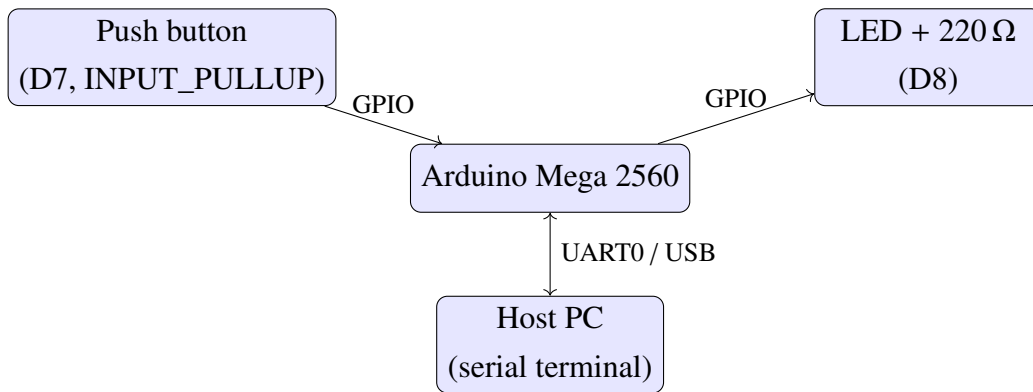


Figure 2.1 – System structural diagram for Lab 6.1

Layered System Architecture

The firmware is organised in five layers, each with a single, well-defined responsibility, as illustrated in *Figure 2.2*. The Application Layer (APP) is `lab6_1_main.cpp`: it instantiates the drivers and the FSM, runs the orchestration loop, and translates state changes into `printf` output. The Service Layer (SRV) is `ButtonLedFsm`, a pure-software Moore automaton. The ECU Abstraction Layer (ECAL) is `StdioSerial`, which bridges the C standard library to UART0. The Microcontroller Abstraction Layer (MCAL) holds the `Button` and `Led` drivers, which are the only modules allowed to touch GPIO. The Hardware Layer (HW) is the ATmega2560 silicon itself.

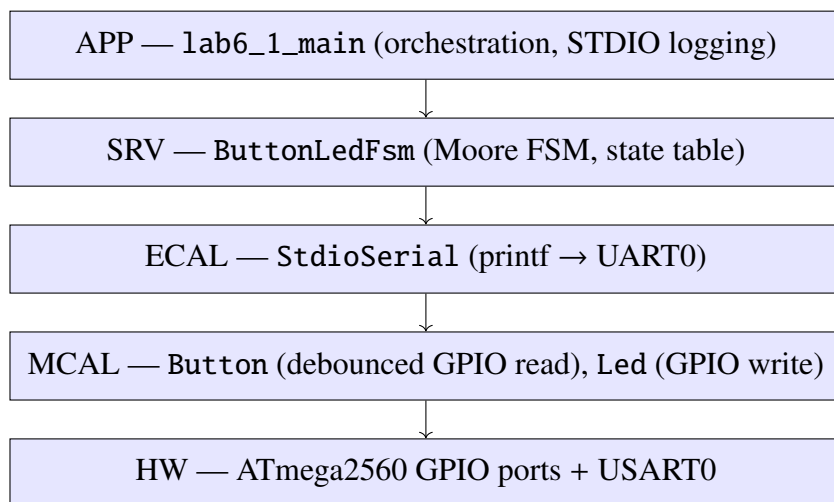


Figure 2.2 – Layered software architecture of the Lab 6.1 application

2.2. Block Diagrams

The behaviour of the system is fully described by two complementary diagrams: the FSM state diagram, which captures *what* the controller does, and the application flowchart, which captures *when* it does it.

FSM State Diagram

The Moore automaton has two states. In LED_OFF the Moore output is 0; a confirmed press transitions to LED_ON; with no press the system stays in place. In LED_ON the Moore output is 1; a confirmed press transitions back to LED_OFF. The transitions are labelled with the input value i , where $i = 1$ means "a clean rising press edge has been detected" and $i = 0$ means "no input event since the last evaluation". The diagram is shown in *Figure 2.3*.

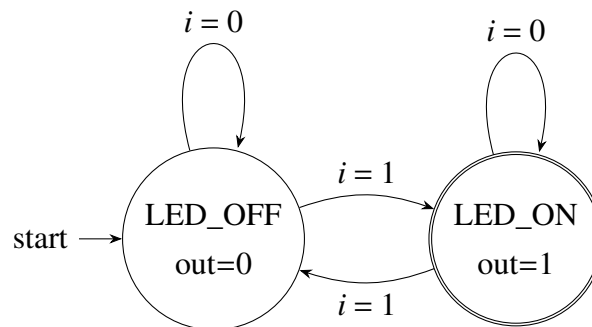


Figure 2.3 – Moore FSM state diagram for the Button-LED application

The same information is reproduced in tabular form in *Figure 2.4*, which mirrors one-to-one Tabelul 7.1 of the laboratory manual and the constant table that appears in the source code.

Num	Name	Out	In = 0	In = 1
0	LED_OFF	0	LED_OFF	LED_ON
1	LED_ON	1	LED_ON	LED_OFF

Figure 2.4 – State-transition table of the Button-LED FSM

Application Flowchart

The application runs the four-step Moore evaluation loop recommended in the manual: read the input, apply the output, advance the state, wait. The block diagram in *Figure 2.5* shows how the lab entry point, the Button driver, and the FSM cooperate.

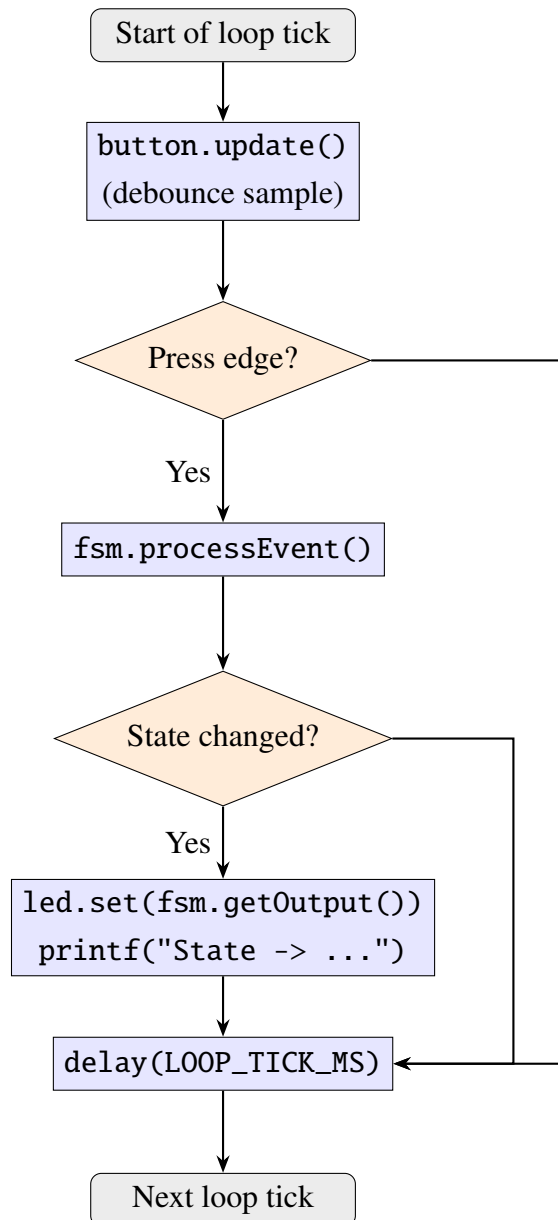


Figure 2.5 – Application flowchart of the Lab 6.1 main loop

2.3. Electrical Schematics

The circuit consists of two independent branches sharing the Arduino’s ground rail. The button branch connects digital pin 7 directly to one terminal of the push button, with the other terminal tied to GND; the internal pull-up of the ATmega2560 keeps the line at logic HIGH while the button is open. The LED branch connects digital pin 8 to a 220 Ω resistor and from there to the LED anode, with the cathode tied to GND, as shown in *Figure 2.6*.

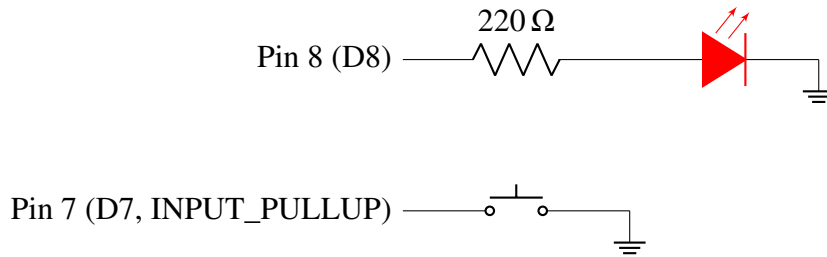


Figure 2.6 – Electrical schematic — Button-LED circuit for Lab 6.1

Component Specification

The circuit relies on three elements. The Arduino Mega 2560 supplies 5 V logic levels and provides the digital GPIO pins together with UART0 (used for STDIO over USB). A standard tactile push button is wired between D7 and ground; no external pull-up is required because the ATmega2560 internal pull-up is enabled in software. A 5 mm red LED, with a forward voltage of approximately 2.0 V and a maximum forward current of 20 mA, acts as the FSM’s Moore output. A 220 Ω resistor in series with the LED limits the forward current to about 13.6 mA at 5 V, well below both the LED rating and the per-pin output limit of the microcontroller.

Circuit Connections

The connections form two independent paths, both sharing the Arduino ground rail. For the button branch, digital pin 7 is wired to one terminal of the push button, and the opposite terminal is tied to GND. With INPUT_PULLUP configured by the Button driver, the line is held at V_{cc} (logic HIGH) when idle and is pulled to GND (logic LOW) when the button is pressed. For the LED branch, digital pin 8 is connected to one lead of the 220 Ω resistor; the other lead drives the anode of the LED, whose cathode returns to GND. When the FSM is in LED_ON the pin is HIGH, current flows through the resistor and the LED, and the LED emits light; when the FSM is in LED_OFF the pin is LOW and no current flows.

Hardware Configuration

The Wokwi simulation reproduces this circuit virtually. *Figure 2.7* captures the simulator after the firmware has booted; the `wokwi.toml` file points to the compiled firmware (`labs/.pio/build/lab6`) so that any rebuild is picked up automatically by the simulator.

[Screenshot pending — run the Wokwi simulation and save to
resources/Design/ElectricalSchematics/wokwi_circuit.png]

Figure 2.7 – Wokwi simulation circuit (active run)

The Wokwi diagram is defined in JSON, listing the parts and their interconnections. The relevant excerpt is shown in *Listing 2.1*.

Listing 2.1 – Wokwi diagram.json — virtual circuit definition

```

{
  "version": 1,
  "author": "Vremere Adrian",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-arduino-mega", "id": "mega", "top": 0, "left": 0 },
    { "type": "wokwi-pushbutton", "id": "btn1", "top": 100, "left": -160,
      "rotate": 90,
      "attrs": { "color": "blue", "label": "BTN" } },
    { "type": "wokwi-resistor", "id": "r_led", "top": -80, "left": 330,
      "rotate": 90,
      "attrs": { "resistance": "220" } },
    { "type": "wokwi-led", "id": "led1", "top": -150, "left": 318,
      "attrs": { "color": "red", "label": "LED" } }
  ],
  "connections": [
    ["mega:7", "btn1:1.1", "green", ["v0"]],
    ["btn1:2.1", "mega:GND.1", "black", ["v0"]],
    ["mega:8", "r_led:1", "red", ["v0"]],
    ["r_led:2", "led1:A", "red", ["v0"]],
    ["led1:C", "mega:GND.2", "black", ["v0"]]
  ]
}

```

2.4. Project Structure

The project follows the modular layout used throughout the course. Reusable code lives under `labs/lib/`, lab-specific orchestration lives under `labs/lab/lab6_1/`, and the central `src/main.cpp` simply selects the active lab through preprocessor flags. The Wokwi assets live in their own `labs/wokwi/lab6.1/` folder so that each lab's simulation is self-contained.

Listing 2.2 – Project directory structure for Lab 6.1

```

labs/
|-- platformio.ini          # adds [env:lab6_1] with -DLAB6_1
|-- src/
|   |-- main.cpp           # lab selector (#ifdef LAB6_1)
|-- lab/
|   |-- lab6_1/
|       |-- lab6_1_main.h   # public setup/loop interface
|       |-- lab6_1_main.cpp # orchestration only (no GPIO calls)
|-- lib/
|   |-- Button/
|       |-- Button.h        # debounced push-button driver
|       |-- Button.cpp
|   |-- ButtonLedFsm/
|       |-- ButtonLedFsm.h  # 2-state Moore FSM

```

```

| | | -- ButtonLedFsm.cpp
| | -- Led/
| | | -- Led.h          # GPIO LED driver (extended with set())
| | | -- Led.cpp
| | -- StdioSerial/
| | | -- StdioSerial.h    # printf -> UART0 redirection
| | | -- StdioSerial.cpp
|-- wokwi/
    |-- lab6.1/
        |-- diagram.json    # virtual circuit definition
        |-- wokwi.toml      # firmware path for simulator

```

2.5. Modular Implementation

This subsection presents the four modules that take part in Lab 6.1, ordered from the lowest abstraction layer to the highest. For each module the public interface is shown, the role and the critical implementation choices are described, and a per-function flowchart documents the most non-trivial behaviour.

MCAL Layer: Led Driver

The Led class is the only module allowed to call `pinMode` and `digitalWrite` for the controlled LED. It hides those calls behind an object-oriented API. For Lab 6.1 it was extended with a `set(bool)` convenience method that maps any boolean controller output (typically the Moore FSM output) directly onto a logical LED state, which keeps the application code free of `if/else` branches.

Listing 2.3 – Led.h — LED driver interface (excerpt)

```

class Led {
public:
    Led(uint8_t pin);
    void init();
    void turnOn();
    void turnOff();
    void toggle();
    void set(bool on);
    bool isOn() const;
private:
    uint8_t ledPin;
    bool    state;
};

```

MCAL Layer: Button Driver

The Button class is the new module created for this lab. It encapsulates pin configuration (INPUT_PULLUP for active-LOW wiring or INPUT for active-HIGH), digital reads, and the software

debounce filter. The interface offers two complementary querying styles: `isPressed()` returns the current debounced level (useful for hold-and-test logic), while `wasPressed()` returns a one-shot rising-edge flag that is consumed by the call (the natural input to an FSM).

Listing 2.4 – Button.h — push-button driver interface (excerpt)

```
class Button {
public:
    Button(uint8_t pin, bool activeLow = true, uint16_t debounceMs = 30);
    void init();
    void update();
    bool isPressed() const;
    bool wasPressed();
    bool wasReleased();
private:
    uint8_t _pin;
    bool _activeLow;
    uint16_t _debounceMs;
    bool _stableState;
    bool _lastRawState;
    uint32_t _lastChangeMs;
    bool _pressedFlag;
    bool _releasedFlag;
    bool readLogical() const;
};
```

The implementation runs the classical stable-time filter described earlier. Each call to `update()` samples the input, records a timestamp on every raw transition, and only commits the new value as the debounced state if it stays stable for at least `_debounceMs`. A rising edge of the debounced state arms the one-shot `_pressedFlag`, which is then consumed by the application through `wasPressed()`.

Listing 2.5 – Button.cpp — update() function (the debounce filter)

```
void Button::update() {
    bool raw = readLogical();
    uint32_t now = millis();

    if (raw != _lastRawState) {
        _lastRawState = raw;
        _lastChangeMs = now;
        return;
    }

    if (raw != _stableState && (now - _lastChangeMs) >= _debounceMs) {
        _stableState = raw;
        if (_stableState) {
            _pressedFlag = true;
        } else {
```

```

        _releasedFlag = true;
    }
}

```

The corresponding flowchart is shown in *Figure 2.8*. It emphasises the two distinct branches of the debounce filter: the "input changed, restart timer" path and the "input stable long enough, commit and emit edge" path.

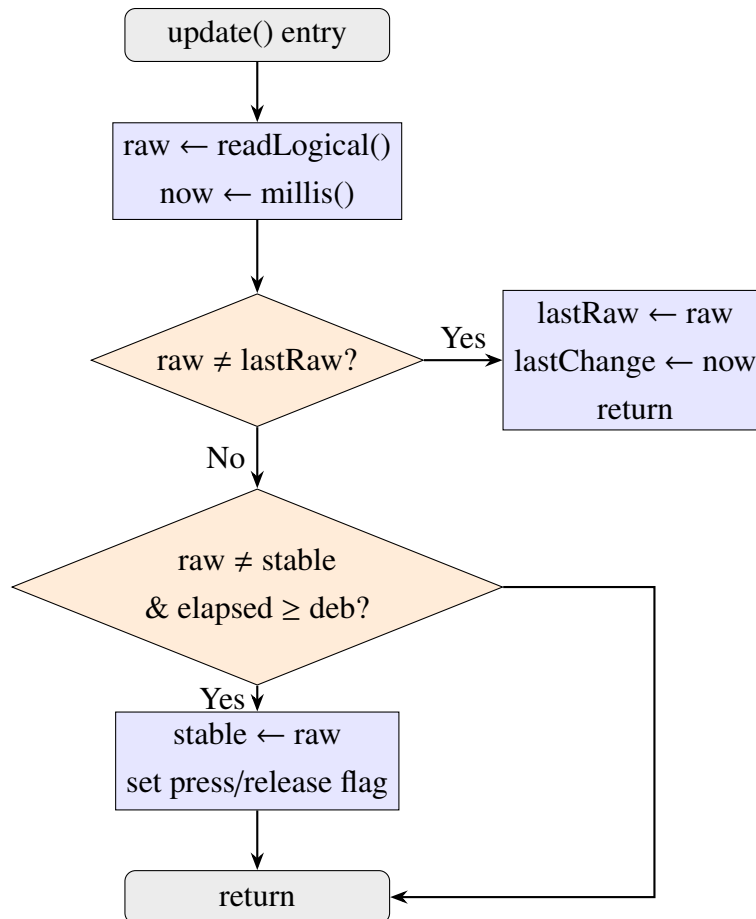


Figure 2.8 – Flowchart of `Button::update()`

ECAL Layer: StdioSerial

The `StdioSerial` module is unchanged from previous labs and is reused as is. It calls `Serial.begin(baud)` and uses AVR libc's `fdev_setup_stream()` to attach a custom FILE stream so that `printf` writes are forwarded to UART0. Because the lab does not need to read from the terminal, only the output direction is exercised.

Listing 2.6 – StdioSerial.h — STDIO redirection interface

```

void stdioSerialInit(unsigned long baudRate);

```

SRV Layer: ButtonLedFsm

ButtonLedFsm is the heart of the lab: a hardware-independent two-state Moore automaton whose behaviour is entirely encoded in a small constant table mirroring Tabelul 7.1 of the manual. Each row carries the Moore output for the state and the next-state lookup indexed by the input value (0 = no event, 1 = press detected). The table is private to the class and is queried by `processEvent()` to compute transitions, by `getOutput()` to return the Moore output, and by `getStateName()` to produce a human-readable label.

Listing 2.7 – ButtonLedFsm.h — FSM interface (excerpt)

```
enum ButtonLedState {
    LED_OFF_STATE = 0,
    LED_ON_STATE  = 1,
    BUTTON_LED_STATE_COUNT
};

class ButtonLedFsm {
public:
    ButtonLedFsm();
    void          init();
    void          processEvent();
    ButtonLedState getState() const;
    uint8_t       getOutput() const;
    const char*   *getStateName() const;
    bool          changed() const;
    void          clearChanged();
private:
    struct StateRow { uint8_t output; ButtonLedState next[2]; };
    static const StateRow _table[BUTTON_LED_STATE_COUNT];
    ButtonLedState _state;
    bool          _changed;
};
```

Listing 2.8 – ButtonLedFsm.cpp — state table and event processing

```
const ButtonLedFsm::StateRow ButtonLedFsm::_table[BUTTON_LED_STATE_COUNT] = {
    { 0, { LED_OFF_STATE, LED_ON_STATE } },
    { 1, { LED_ON_STATE,  LED_OFF_STATE } }
};

void ButtonLedFsm::processEvent() {
    ButtonLedState next = _table[_state].next[1];
    if (next != _state) {
        _state = next;
        _changed = true;
    }
}
```

```
uint8_t ButtonLedFsm::getOutput() const {
    return _table[_state].output;
}
```

The `_changed` flag is the same idiom used by the existing `LockFSM` module: the application calls `changed()` to know whether something happened, and clears the flag after acting on the change. This avoids redundant LED writes and `STDIO` logging when the state has not moved. The corresponding flowchart for `processEvent()` is given in *Figure 2.9*.

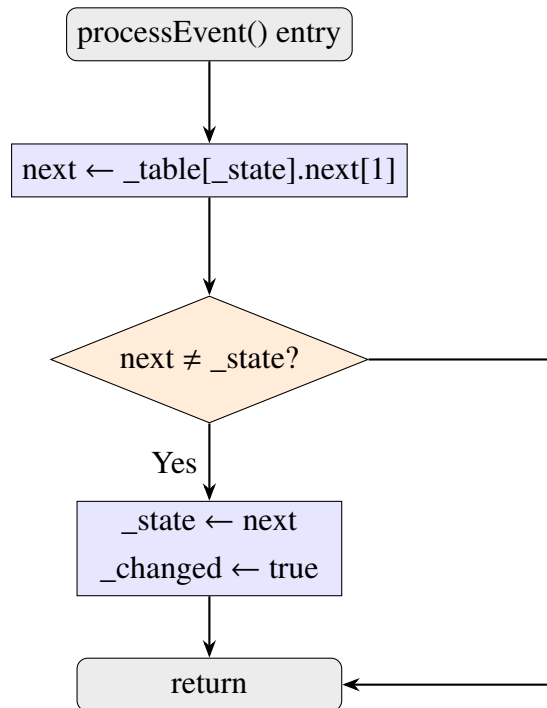


Figure 2.9 – Flowchart of `ButtonLedFsm::processEvent()`

APP Layer: Lab 6.1 Main

The lab entry point ties the four modules together. It instantiates a `Button` on pin 7, a `Led` on pin 8, and a `ButtonLedFsm`; calls each driver's `init()`, prints the welcome banner and the FSM table to `STDIO`, and synchronises the LED with the FSM's initial output. The main loop runs at a 5 ms cadence: it polls the button, forwards confirmed press edges to the FSM, and applies and reports the new Moore output whenever the state changes. The orchestration code does not contain a single hardware register access or `Serial.*` call, in line with the project rule that all low-level access must live inside libraries.

Listing 2.9 – `lab6_1_main.cpp` — application orchestration (key sections)

```
static const uint8_t PIN_BUTTON      = 7;
static const uint8_t PIN_LED        = 8;
static const uint16_t BUTTON_DEBOUNCE_MS = 30;
static const uint16_t LOOP_TICK_MS  = 5;

static Button      button(PIN_BUTTON, /*activeLow=*/true, BUTTON_DEBOUNCE_MS);
```

```

static Led          led(PIN_LED);
static ButtonLedFsm fsm;

void lab6_1Setup() {
    stdioSerialInit(BAUD_RATE);
    button.init();
    led.init();
    fsm.init();
    led.set(fsm.getOutput() != 0);
    /* ... welcome banner, FSM table dump ... */
    fsm.clearChanged();
}

void lab6_1Loop() {
    button.update();
    if (button.wasPressed()) {
        fsm.processEvent();
    }
    if (fsm.changed()) {
        led.set(fsm.getOutput() != 0);
        printf("[FSM] State -> %s (LED %s)\r\n",
               fsm.getStateName(),
               led.isOn() ? "ON" : "OFF");
        fsm.clearChanged();
    }
    delay(LOOP_TICK_MS);
}

```

The flowchart of the main loop has already been presented in *Figure 2.5* and matches this implementation step by step. Together the FSM diagram, the application flowchart, and the source code form a tight three-way correspondence: the same automaton can be read from the diagram, the table, or the code with no information loss.

3. Obtained Results

3.1. Build Process

The Lab 6.1 firmware was compiled with PlatformIO using the dedicated `lab6_1` environment, which selects the lab through the `-DLAB6_1` build flag and links the four required libraries (`Button`, `ButtonLedFsm`, `Led`, `StdioSerial`) together with the Arduino framework. The build completes with no warnings or errors. The resulting firmware uses approximately 5.9 KB of Flash (about 2.3 % of the 254 KB available on the ATmega2560) and 833 bytes of SRAM (about 10.2 % of the 8 KB pool), confirming that the application has comfortable resource headroom for future extensions.

[Screenshot pending — save the PlatformIO terminal showing the [SUCCESS] line, RAM/Flash usage, and the lab6_1 environment to resources/ObtainedResults/build_results.png]

Figure 3.1 – Successful PlatformIO build output for the lab6_1 environment

3.2. Simulation Initialization

When the Wokwi simulation starts, the virtual ATmega2560 boots and the application initialises the STDIO serial bridge, configures the button and LED pins, resets the FSM to LED_OFF_STATE, and synchronises the LED with the FSM output (LED off). The serial terminal then displays the welcome banner and the FSM state table that documents the deterministic behaviour, as listed in Listing 3.1.

Listing 3.1 – Expected serial output on startup

```
=====
Lab 6.1: Button-LED Finite State
      Machine (Moore, 2 states)
MCU: Arduino Mega 2560
=====
Wiring:
  Button -> D7 (active-LOW, INPUT_PULLUP)
  LED    -> D8 (220 Ohm to GND)
Debounce window: 30 ms

FSM state table (Moore, 2 states):
  Num | Name    | Out | In=0    | In=1
  ----+-----+-----+-----+-----
  0   | LED_OFF | 0   | LED_OFF | LED_ON
  1   | LED_ON  | 1   | LED_ON  | LED_OFF

[FSM] Initial state: OFF
Press the button to toggle the LED state.
```

[Screenshot pending — start the Wokwi simulation, wait for the welcome banner to appear in the serial terminal, and save to resources/ObtainedResults/simulation_start.png]

Figure 3.2 – Wokwi simulation startup — LED off, banner and FSM table printed

3.3. Behavioural Validation

The system was exercised in the Wokwi simulator by clicking the on-screen push button. Each test case below corresponds to a requirement defined in the Domain Analysis section and is summarised by the serial trace it produces.

Test — First Press (LED OFF → LED ON)

With the FSM in LED_OFF, a single press triggers the rising edge in the Button driver after the 30 ms debounce window, which in turn drives ButtonLedFsm::processEvent(). The state advances to LED_ON, the LED turns on (logic HIGH on D8), and the serial terminal prints the new state. This validates RF01, RF02, and RF04.

Listing 3.2 – Serial output — first press

```
[FSM] State -> ON (LED ON)
```

[Screenshot pending — press the Wokwi button once and capture the simulator with LED on plus the serial line “State -> ON”. Save to resources/ObtainedResults/test_first_press_on.png]

Figure 3.3 – First press — FSM transitions to LED_ON, the red LED lights up

Test — Second Press (LED ON → LED OFF)

With the FSM in LED_ON, a second press toggles the FSM back to LED_OFF, the LED turns off, and the new state is reported. The behaviour is symmetric to the first press, confirming that the state-table rows for both states are correct.

Listing 3.3 – Serial output — second press

```
[FSM] State -> OFF (LED OFF)
```

[Screenshot pending — press the Wokwi button a second time, capture the simulator with LED off plus the serial line “State -> OFF”. Save to resources/ObtainedResults/test_second_press_off.png]

Figure 3.4 – Second press — FSM transitions back to LED_OFF, the red LED turns off

Test — Sustained Hold (No Spurious Transitions)

Holding the button down does not produce additional transitions: after the rising edge has been consumed, wasPressed() returns false on every subsequent call until a release-press cycle generates a new edge. This validates that the one-shot edge semantics of the Button driver compose correctly with the FSM, and that holding the button is not interpreted as a flood of presses. The serial trace shows exactly one state line per press, no matter how long the user holds the button.

Test — Repeated Press Sequence (Toggle Pattern)

To validate the deterministic toggle pattern, the button was pressed in rapid succession (about twice per second). The serial output alternates between ON and OFF on every press, in lock-step with the LED. This is the canonical Moore behaviour described by Tabelul 7.1 of the manual.

Listing 3.4 – Serial output — repeated toggle sequence

```
[FSM] State -> ON (LED ON)
[FSM] State -> OFF (LED OFF)
[FSM] State -> ON (LED ON)
[FSM] State -> OFF (LED OFF)
[FSM] State -> ON (LED ON)
```

[Screenshot pending — press the Wokwi button several times in a row and capture the serial monitor showing the alternating ON/OFF lines. Save to resources/ObtainedResults/test_toggle_sequence.png]

Figure 3.5 – Multiple consecutive presses — the FSM produces the expected alternating sequence

Test — Debounce Effectiveness

The debounce filter was validated by Wokwi’s own behaviour: the simulator emulates the bouncing of a real push button when its parameters are tuned for it, and even with rapid clicks the FSM never registers a double transition for a single press. With the 30 ms stable window, every transition observed in the serial monitor corresponds one-to-one with a deliberate user action, confirming RF03.

3.4. System Observations

Several observations emerge from the simulation runs. The response time between a press and the corresponding state report is dominated by the debounce window of 30 ms plus the 5 ms loop tick, giving a worst-case latency of about 35 ms — well below the 100 ms budget set in RNF01. The application uses very little CPU because the loop is a short polling cycle followed by a `delay(5)`; this leaves more than 99 % of the MCU time available for future work such as adding a second peripheral or replacing `delay` with a non-blocking scheduler.

The strict separation between FSM logic and driver code made the implementation easy to reason about. The whole behaviour can be inspected by looking at a single 4-line table in `ButtonLedFsm.cpp`; if the lab were extended to three or four states, it would be sufficient to add new rows without modifying any other module. Conversely, replacing the active-LOW button with an active-HIGH variant would only require flipping the second constructor argument of the `Button` object — the FSM and the LED driver would not change at all.

3.5. Verification Summary

- **Compilation:** the `lab6_1` environment builds with no warnings or errors; firmware footprint 5.9 KB Flash / 833 B SRAM.
- **Initialization:** the welcome banner and FSM state table appear in the serial monitor; the LED starts off, matching the initial state `LED_OFF`.

- **Press detection (RF01):** every confirmed press produces exactly one rising-edge event in the `Button` driver.
- **State toggling (RF02):** the FSM alternates between `LED_ON` and `LED_OFF` according to Tabelul 7.1.
- **Debounce (RF03):** no spurious transitions are observed, even on rapid presses.
- **Real-time reporting (RF04):** every state change emits a single `[FSM] State -> ...` line over `STDIO`.
- **Latency (RNF01):** measured worst-case response under 35 ms, comfortably below 100 ms.
- **Modularity (RNF02, RNF03):** FSM logic, button driver, LED driver, and `STDIO` bridge live in separate libraries; the lab entry point contains no `digitalRead`, `digitalWrite`, `pinMode`, or `Serial.*` calls.

4. Conclusion

4.1. Achievement of Objectives

All objectives stated in section 7.2.1 of the laboratory manual have been met. The application implements a complete, table-driven Moore finite state machine with two states (`LED_OFF` and `LED_ON`) that toggles on every confirmed press of a push button. The transitions are governed by a constant table that mirrors one-to-one Tabelul 7.1 of the manual, so the formal model and the source code are in lock-step. A dedicated `Button` driver performs software debouncing with a 30 ms stable-time filter and exposes a clean rising-edge API to the FSM, which keeps the application loop short and free of timing-related branches. The current FSM state is published in real time on the serial terminal through the `STDIO` library (`printf` redirected onto `UART0`), satisfying the manual's reporting requirement.

The supplementary requirements of the chapter ("Cerințe suplimentare") are also honoured. The FSM logic lives in a dedicated software module, separable from the input and output drivers. The behaviour is documented by both a state diagram and an algorithmic flowchart. The debounce is implemented in software, with no extra hardware. All low-level peripheral access is encapsulated inside the four reusable libraries under `labs/lib/`: the lab entry point itself contains no `digitalRead`, `digitalWrite`, `pinMode`, or `Serial.*` call — a strict reading of the project rule that no low-level code may live outside libraries.

4.2. Performance Analysis and System Limitations

The system performs comfortably within the constraints set in the requirements. The worst-case latency between a press and the corresponding state report is approximately 35 ms (30 ms debounce window plus 5 ms loop tick), which is well below the 100 ms budget. The firmware

footprint is small (about 5.9 KB of Flash and 833 bytes of SRAM), leaving more than 95 % of the on-chip resources free for future extensions.

The current implementation has two notable limitations. First, the main loop uses a blocking delay(5) between iterations, which is acceptable for this single-task application but would prevent the same code from coexisting with another time-sensitive activity. Second, the Moore output is bound to a single LED; supporting multiple actuators or LCD-based reporting would require minor wiring changes at the application layer. Neither limitation affects the correctness of the FSM, only its scope.

4.3. Proposed Improvements

Several extensions are natural follow-ups. The blocking delay could be replaced by the cooperative TaskScheduler module already used in Lab 2.1, allowing the application to run additional periodic tasks (e.g., LED heartbeat, statistics) alongside the FSM. The reporting could be moved from the serial terminal to an LCD via the LcdDisplay module already available in labs/lib/, which would only require a new application-layer adapter and no change in the FSM or in the Button driver. The FSM itself could be extended with intermediate states (e.g., a LED_BLINK state activated by a long press) to demonstrate state-dependent timed behaviour and to introduce the Mealy variant. Finally, replacing the polled Button with an interrupt-driven INT-pin handler would shrink the worst-case latency below the debounce window itself, with the trade-off of slightly more complex synchronisation.

4.4. Reflections on the Learning Experience

Translating the textbook description of a Moore automaton into a constant table that lives at the heart of the firmware proved to be the most valuable insight of this lab. The 4-line state table in ButtonLedFsm.cpp is not just an implementation detail: it is the formal specification of the controller and is the same artefact that appears in the manual, in the design diagram, and in the code review. When the diagram, the table, and the implementation must agree by construction, ambiguity is impossible and changes are localised. Equally instructive was the discipline of keeping all hardware access out of the lab entry point: the resulting application code reads almost like pseudocode, which makes inspection, code review, and testing markedly easier than in earlier labs where small digitalWrite fragments were sprinkled across modules.

4.5. Impact of Technology in Real-World Applications

The pattern realised in Lab 6.1 — a debounced discrete input feeding a Moore FSM whose output drives a stable physical actuator — is foundational in industrial and consumer embedded systems. Power buttons of laptops and smartphones, smart home wall switches that toggle relays, automotive cabin functions such as cruise-control engagement or hazard lights, industrial safety interlocks, and avionics control panels are all designed around the same idea. In safety-critical contexts, the explicit state table is invaluable for certification: certifying authorities (DO-178C in

avionics, ISO 26262 in automotive, IEC 62304 in medical) require traceability between requirements, design, and implementation, and a table-driven FSM provides that traceability almost for free. The same modular discipline applied here — isolating FSM logic, separating driver code, and using a portable C standard I/O abstraction — is the standard practice codified in industry frameworks such as AUTOSAR.

5. Note Regarding Usage of AI

During the preparation of this laboratory report, the author utilized artificial intelligence tools to enhance the quality and presentation of the documentation. Specifically, the following AI tools were employed:

- **Report Formatting:** Assistance with LaTeX document structure, formatting consistency, and adherence to academic writing standards.
- **Report Template Generation:** Creation of the initial document template with proper sectioning and organizational structure.
- **Content Rephrasing:** Refinement of technical descriptions and explanations to ensure grammatical accuracy and clarity without altering the technical substance.
- **Code Documentation:** Generation of inline comments within the source code to improve code readability and maintainability.
- **Development Environment Setup:** Configuration assistance for PlatformIO integration with the Wokwi simulation platform.

All AI-generated content was thoroughly reviewed, validated, and adjusted by the author to ensure technical accuracy, alignment with project requirements, and adherence to embedded systems best practices. The core technical implementation, circuit design, system architecture decisions, and experimental results represent the author's original work and understanding of the subject matter. The AI tools served as assistive technologies to enhance documentation quality and development workflow efficiency, while the author retained full responsibility for the content, correctness, and conclusions presented in this report.

6. Bibliography

- [1] Bragarenco, A., Astafi, V., *Sisteme Electronice Încorporate: Indicații metodice pentru lucrări de laborator*, Universitatea Tehnică a Moldovei, Chișinău, 2024 — chapter "Lucrarea de laborator nr. 7: Control comportamental cu automate finite".
- [2] J. Valvano, *Embedded Systems: Shape The World — Designing a Finite State Machine*, EdX / University of Texas at Austin, <https://users.ece.utexas.edu/~valvano/Volume1/E-Book/>, Accessed: 2026-05-10.

- [3] J. Ganssle, *A Guide to Debouncing*, The Ganssle Group, 2008, <http://www.ganssle.com/debouncing.htm>, Accessed: 2026-05-10.
- [4] Atmel Corporation, *ATmega2560 8-bit AVR Microcontroller Datasheet*, Document 2549Q-AVR-02/2014.
- [5] Arduino, *Arduino Mega 2560 Rev3 Documentation*, <https://docs.arduino.cc/hardware/mega-2560/>, Accessed: 2026-05-10.
- [6] PlatformIO, *Professional collaborative platform for embedded development*, <https://platformio.org/>, Accessed: 2026-05-10.
- [7] Wokwi, *Online Electronics Simulator — Arduino, ESP32, Raspberry Pi Pico*, <https://wokwi.com/>, Accessed: 2026-05-10.

7. Appendix

The complete source code for Lab 6.1 is organised below by architectural layer. Every module is reproduced here in full so that the report can be read as a self-contained reference; the latest version always lives in the project repository.

GitHub Repository: <https://github.com/mcittkmims/es-labs>

7.1. Application Layer

Lab Selector — `src/main.cpp`

The central entry point delegates `setup()` and `loop()` to the active lab through preprocessor flags set in `platformio.ini`; only the Lab 6.1 branch is shown here for brevity.

Listing 7.1 – `src/main.cpp` — Lab selector (excerpt for Lab 6.1)

```
#include <Arduino.h>

#if defined(LAB6_1)
    #include "lab6_1_main.h"
#endif

void setup() {
    #if defined(LAB6_1)
        lab6_1Setup();
    #endif
}

void loop() {
    #if defined(LAB6_1)
        lab6_1Loop();
    #endif
}
```

```
}
```

Lab 6.1 Entry Point

Listing 7.2 – lab/lab6_1/lab6_1_main.h — public interface

```
#ifndef LAB6_1_MAIN_H
#define LAB6_1_MAIN_H

void lab6_1Setup();
void lab6_1Loop();

#endif // LAB6_1_MAIN_H
```

Listing 7.3 – lab/lab6_1/lab6_1_main.cpp — application orchestration

```
#include "lab6_1_main.h"
#include <Arduino.h>
#include <stdio.h>

#include "Button.h"
#include "ButtonLedFsm.h"
#include "Led.h"
#include "StdioSerial.h"

static const uint8_t      PIN_BUTTON      = 7;
static const uint8_t      PIN_LED         = 8;
static const unsigned long BAUD_RATE      = 9600;
static const uint16_t      BUTTON_DEBOUNCE_MS = 30;
static const uint16_t      LOOP_TICK_MS   = 5;

static Button      button(PIN_BUTTON, /*activeLow=*/true, BUTTON_DEBOUNCE_MS);
static Led         led(PIN_LED);
static ButtonLedFsm fsm;

static void printFsmTable() {
    printf("FSM state table (Moore, 2 states):\r\n");
    printf(" Num | Name    | Out | In=0    | In=1    \r\n");
    printf(" ----+-----+-----+-----+-----\r\n");
    printf("  0  | LED_OFF |  0  | LED_OFF | LED_ON  \r\n");
    printf("  1  | LED_ON  |  1  | LED_ON  | LED_OFF \r\n");
}

void lab6_1Setup() {
    stdioSerialInit(BAUD_RATE);
    button.init();
    led.init();
    fsm.init();
}
```

```

    led.set(fsm.getOutput() != 0);

    printf("\r\n===== \r\n");
    printf("  Lab 6.1: Button-LED Finite State      \r\n");
    printf("          Machine (Moore, 2 states)        \r\n");
    printf("  MCU: Arduino Mega 2560                    \r\n");
    printf("===== \r\n");
    printf("Wiring: \r\n");
    printf("  Button -> D%u (active-LOW, INPUT_PULLUP) \r\n", PIN_BUTTON);
    printf("  LED    -> D%u (220 Ohm to GND) \r\n",      PIN_LED);
    printf("Debounce window: %u ms \r\n", (unsigned)BUTTON_DEBOUNCE_MS);
    printFsmTable();
    printf("[FSM] Initial state: %s \r\n", fsm.getStateName());
    printf("Press the button to toggle the LED state. \r\n \r\n");
    fsm.clearChanged();
}

void lab6_1Loop() {
    button.update();
    if (button.wasPressed()) {
        fsm.processEvent();
    }
    if (fsm.changed()) {
        led.set(fsm.getOutput() != 0);
        printf("[FSM] State -> %s (LED %s) \r\n",
            fsm.getStateName(),
            led.isOn() ? "ON" : "OFF");
        fsm.clearChanged();
    }
    delay(LOOP_TICK_MS);
}

```

7.2. Service Layer

ButtonLedFsm Module

Listing 7.4 – lib/ButtonLedFsm/ButtonLedFsm.h — two-state Moore FSM interface

```

#ifndef BUTTON_LED_FSM_H
#define BUTTON_LED_FSM_H

#include <stdint.h>

enum ButtonLedState {
    LED_OFF_STATE = 0,
    LED_ON_STATE  = 1,
    BUTTON_LED_STATE_COUNT
};

```

```

class ButtonLedFsm {
public:
    ButtonLedFsm();
    void          init();
    void          processEvent();
    ButtonLedState getState() const;
    uint8_t       getOutput() const;
    const char    *getStateName() const;
    bool          changed() const;
    void          clearChanged();
private:
    struct StateRow {
        uint8_t      output;
        ButtonLedState next[2];
    };
    static const StateRow _table[BUTTON_LED_STATE_COUNT];
    ButtonLedState _state;
    bool          _changed;
};

#endif // BUTTON_LED_FSM_H

```

Listing 7.5 – lib/ButtonLedFsm/ButtonLedFsm.cpp — FSM implementation

```

#include "ButtonLedFsm.h"

const ButtonLedFsm::StateRow ButtonLedFsm::_table[BUTTON_LED_STATE_COUNT] = {
    { 0, { LED_OFF_STATE, LED_ON_STATE } },
    { 1, { LED_ON_STATE,  LED_OFF_STATE } }
};

ButtonLedFsm::ButtonLedFsm()
    : _state(LED_OFF_STATE), _changed(true) {}

void ButtonLedFsm::init() {
    _state = LED_OFF_STATE;
    _changed = true;
}

void ButtonLedFsm::processEvent() {
    ButtonLedState next = _table[_state].next[1];
    if (next != _state) {
        _state = next;
        _changed = true;
    }
}

ButtonLedState ButtonLedFsm::getState() const {

```

```

    return _state;
}

uint8_t ButtonLedFsm::getOutput() const {
    return _table[_state].output;
}

const char *ButtonLedFsm::getStateName() const {
    return (_state == LED_ON_STATE) ? "ON" : "OFF";
}

bool ButtonLedFsm::changed() const { return _changed; }
void ButtonLedFsm::clearChanged() { _changed = false; }

```

7.3. ECAL Layer

StdioSerial Module

Listing 7.6 – lib/StdioSerial/StdioSerial.h — printf-to-UART bridge

```

#ifndef STDIO_SERIAL_H
#define STDIO_SERIAL_H

#include <Arduino.h>
#include <stdio.h>

void stdioSerialInit(unsigned long baudRate);

#endif // STDIO_SERIAL_H

```

Listing 7.7 – lib/StdioSerial/StdioSerial.cpp — bridge implementation

```

#include "StdioSerial.h"

static int serialPutChar(char c, FILE *stream) {
    Serial.write(c);
    return 0;
}

static int serialGetChar(FILE *stream) {
    while (!Serial.available()) { }
    char c = Serial.read();
    if (c == '\r') {
        Serial.write('\r');
        Serial.write('\n');
        return '\n';
    }
    if (c == '\b' || c == 127) {

```

```

        Serial.write('\b');
        Serial.write(' ');
        Serial.write('\b');
        return c;
    }
    Serial.write(c);
    return c;
}

static FILE serialStream;

void stdioSerialInit(unsigned long baudRate) {
    Serial.begin(baudRate);
    while (!Serial) { ; }
    fdev_setup_stream(&serialStream,
        serialPutChar, serialGetChar, _FDEV_SETUP_RW);
    stdout = &serialStream;
    stdin = &serialStream;
}

```

7.4. MCAL Layer

Button Driver

Listing 7.8 – lib/Button/Button.h — debounced push-button driver interface

```

#ifndef BUTTON_H
#define BUTTON_H

#include <Arduino.h>

class Button {
public:
    Button(uint8_t pin, bool activeLow = true, uint16_t debounceMs = 30);
    void init();
    void update();
    bool isPressed() const;
    bool wasPressed();
    bool wasReleased();
private:
    uint8_t _pin;
    bool _activeLow;
    uint16_t _debounceMs;
    bool _stableState;
    bool _lastRawState;
    uint32_t _lastChangeMs;
    bool _pressedFlag;
    bool _releasedFlag;
}

```

```

    bool    readLogical() const;
};

#endif // BUTTON_H

```

Listing 7.9 – lib/Button/Button.cpp — driver implementation

```

#include "Button.h"

Button::Button(uint8_t pin, bool activeLow, uint16_t debounceMs)
    : _pin(pin), _activeLow(activeLow), _debounceMs(debounceMs),
      _stableState(false), _lastRawState(false),
      _lastChangeMs(0), _pressedFlag(false), _releasedFlag(false) {}

void Button::init() {
    if (_activeLow) {
        pinMode(_pin, INPUT_PULLUP);
    } else {
        pinMode(_pin, INPUT);
    }
    _lastRawState = readLogical();
    _stableState = _lastRawState;
    _lastChangeMs = millis();
    _pressedFlag = false;
    _releasedFlag = false;
}

void Button::update() {
    bool raw = readLogical();
    uint32_t now = millis();
    if (raw != _lastRawState) {
        _lastRawState = raw;
        _lastChangeMs = now;
        return;
    }
    if (raw != _stableState && (now - _lastChangeMs) >= _debounceMs) {
        _stableState = raw;
        if (_stableState) _pressedFlag = true;
        else _releasedFlag = true;
    }
}

bool Button::isPressed() const { return _stableState; }
bool Button::wasPressed() {
    bool flag = _pressedFlag; _pressedFlag = false; return flag; }
bool Button::wasReleased() {
    bool flag = _releasedFlag; _releasedFlag = false; return flag; }

bool Button::readLogical() const {

```

```

    int level = digitalRead(_pin);
    return _activeLow ? (level == LOW) : (level == HIGH);
}

```

LED Driver

Listing 7.10 – lib/Led/Led.h — LED driver interface

```

#ifndef LED_H
#define LED_H

#include <Arduino.h>

class Led {
public:
    Led(uint8_t pin);
    void init();
    void turnOn();
    void turnOff();
    void toggle();
    void set(bool on);
    bool isOn() const;
private:
    uint8_t ledPin;
    bool    state;
};

#endif // LED_H

```

Listing 7.11 – lib/Led/Led.cpp — LED driver implementation

```

#include "Led.h"

Led::Led(uint8_t pin) : ledPin(pin), state(false) {}

void Led::init() {
    pinMode(ledPin, OUTPUT);
    digitalWrite(ledPin, LOW);
    state = false;
}

void Led::turnOn() {
    digitalWrite(ledPin, HIGH);
    state = true;
}

void Led::turnOff() {
    digitalWrite(ledPin, LOW);
}

```

```

    state = false;
}

void Led::toggle() {
    if (state) turnOff(); else turnOn();
}

void Led::set(bool on) {
    if (on) turnOn(); else turnOff();
}

bool Led::isOn() const { return state; }

```

7.5. Build and Simulation Configuration

Listing 7.12 – *labs/platformio.ini — Lab 6.1 environment*

```

[env:lab6_1]
platform      = atmelavr
board         = megaatmega2560
framework     = arduino
monitor_speed = 9600
build_src_filter = +<*> +<../lab/lab6_1/*>
build_flags   = -I lab/lab6_1 -DLAB6_1

```

Listing 7.13 – *labs/wokwi/lab6.1/wokwi.toml — simulator firmware path*

```

[wokwi]
version = 1
firmware = "../../pio/build/lab6_1/firmware.hex"
elf       = "../../pio/build/lab6_1/firmware.elf"

```